

AN DNS

DNS — Section 1: Service Brief

1.1 What is DNS?

DNS (Domain Name System) is a distributed and hierarchical naming system that translates human-readable domain names into IP addresses.

For example:

```
www.example.com → 192.168.10.20
```

DNS also supports reverse resolution:

```
192.168.10.20 → www.example.com
```

1.2 Why DNS is Required

Without DNS, users would need to access network services using IP addresses:

```
http://192.168.10.20
```

With DNS, the same service can be accessed using:

```
http://web.company.local
```

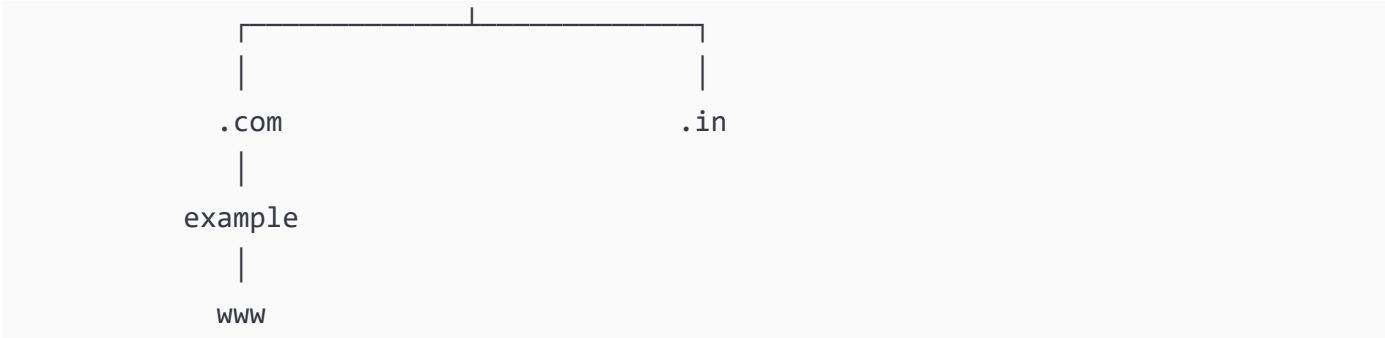
DNS is commonly used for:

- Hostname resolution
- Web services
- Mail-server discovery
- Internal network naming
- Reverse IP-to-hostname resolution
- Internet name resolution

1.3 DNS Hierarchy

DNS uses a hierarchical namespace:

```
Root (.)  
|
```



For:

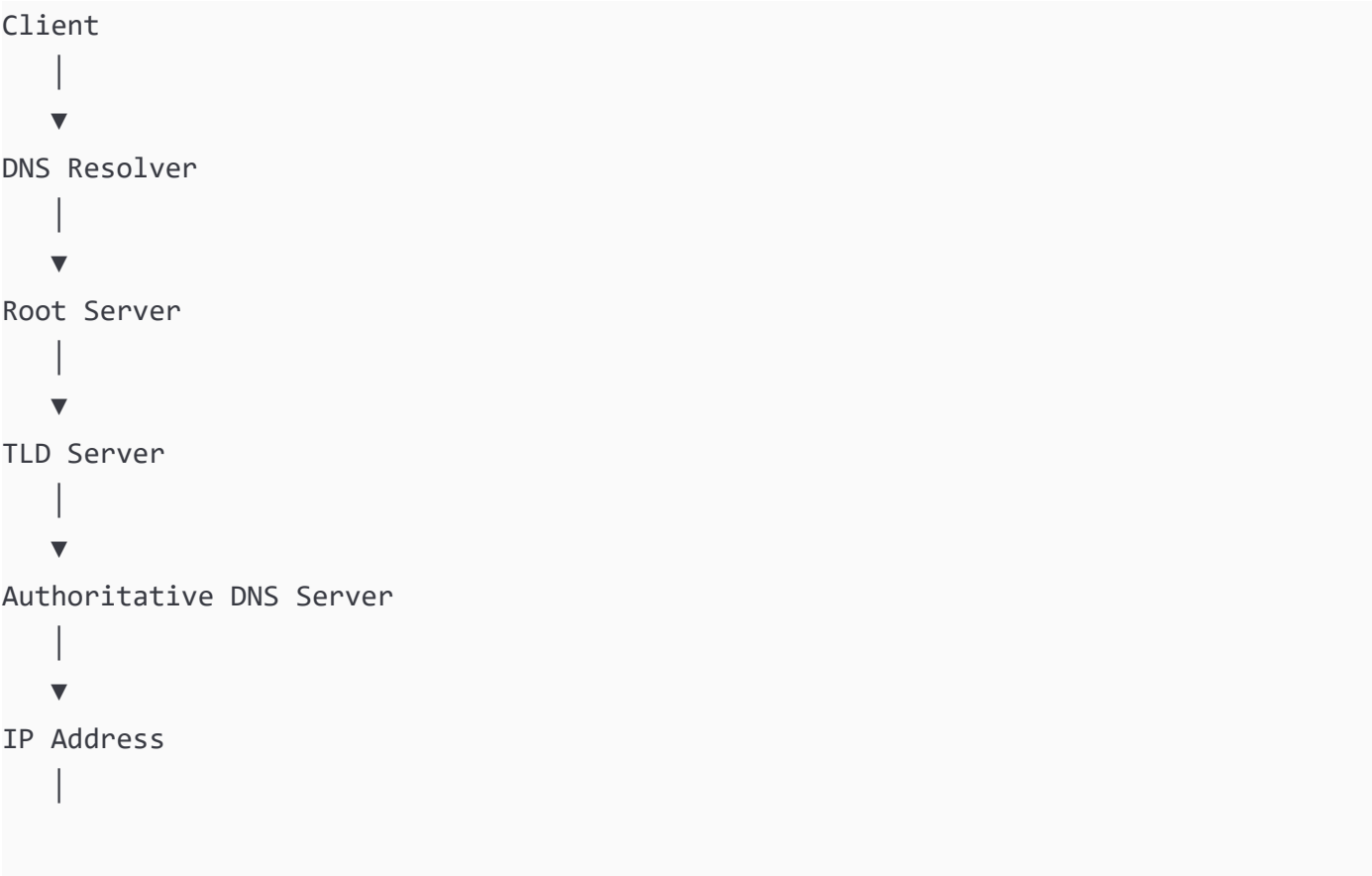
www.example.com

Level	Example	Description
Root	.	Top of DNS hierarchy
TLD	.com	Top-Level Domain
Domain	example	Domain name
Host	www	Specific host/service

1.4 DNS Resolution

DNS resolution is the process of finding the IP address associated with a domain name.

A simplified Internet lookup is:





Client

The resolver may use a cached answer instead of performing the complete lookup again. DNS caching improves response time and reduces repeated DNS queries.

1.5 Forward & Reverse DNS

Forward Lookup

Maps a hostname to an IP address:

Hostname → IP Address

Example:

server.company.local → 192.168.10.10

Common records:

A → IPv4 address

AAAA → IPv6 address

Reverse Lookup

Maps an IP address back to a hostname:

IP Address → Hostname

Example:

192.168.10.10 → server.company.local

The main record used is:

PTR

Forward and reverse zones are configured separately in BIND9.

1.6 DNS Protocol & Port

DNS primarily uses:

Port : 53

Transport : UDP

1.7 Important DNS Records

Record	Purpose	Example
A	Hostname → IPv4	web → 192.168.10.20
AAAA	Hostname → IPv6	web → IPv6 address
PTR	IP → Hostname	192.168.10.20 → web
CNAME	Alias → Hostname	www → web
MX	Specifies mail server	mail.company.local
NS	Specifies name server	dns.company.local
SOA	Zone authority information	Serial, refresh, retry, etc.
TXT	Stores text/policy information	Verification data

1.8 DNS Zone

A **DNS zone** is a portion of the DNS namespace managed by a DNS server.

For our Debian BIND9 implementation:

Forward Zone

company.local

Maps:

Hostname → IP

Reverse Zone

For the network:

192.168.10.0/24

the reverse zone is:

10.168.192.in-addr.arpa

It maps:

IP → Hostname

Example:

Forward:

server.company.local

↓
192.168.10.10

Reverse:
192.168.10.10

↓
server.company.local

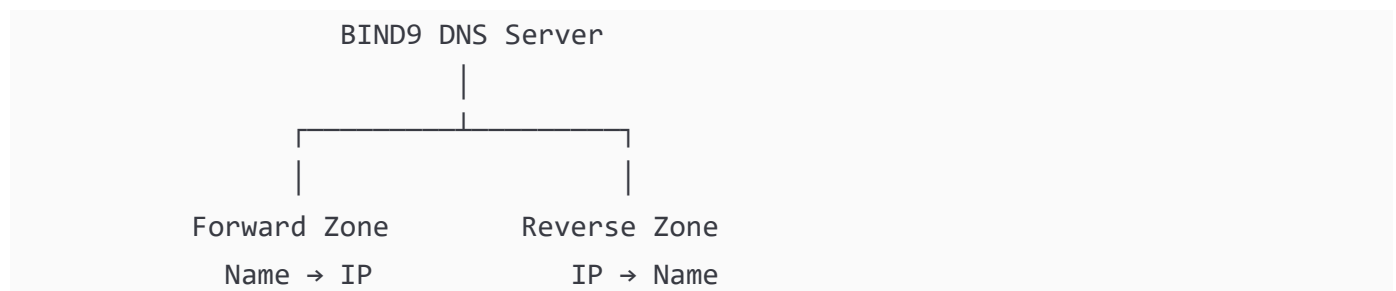
1.9 BIND9

BIND9 (Berkeley Internet Name Domain) is the DNS server software used in our Debian implementation.

BIND9 is responsible for:

- Managing DNS zones
- Maintaining DNS records
- Answering DNS queries
- Providing authoritative DNS responses
- Performing recursive resolution when configured

Basic architecture:



1.10 TTL — Time To Live

TTL (Time To Live) specifies how long a DNS record can normally be cached.

Example:

\$TTL 604800

means:

604800 seconds ≈ 7 days

The TTL value controls how long DNS information can remain cached before it needs to be refreshed.

Section 2: Architecture & Resolution

2.1 DNS Architecture

DNS follows a **hierarchical and distributed architecture**:

Client



DNS Resolver



Root DNS



TLD DNS



Authoritative DNS



IP Address

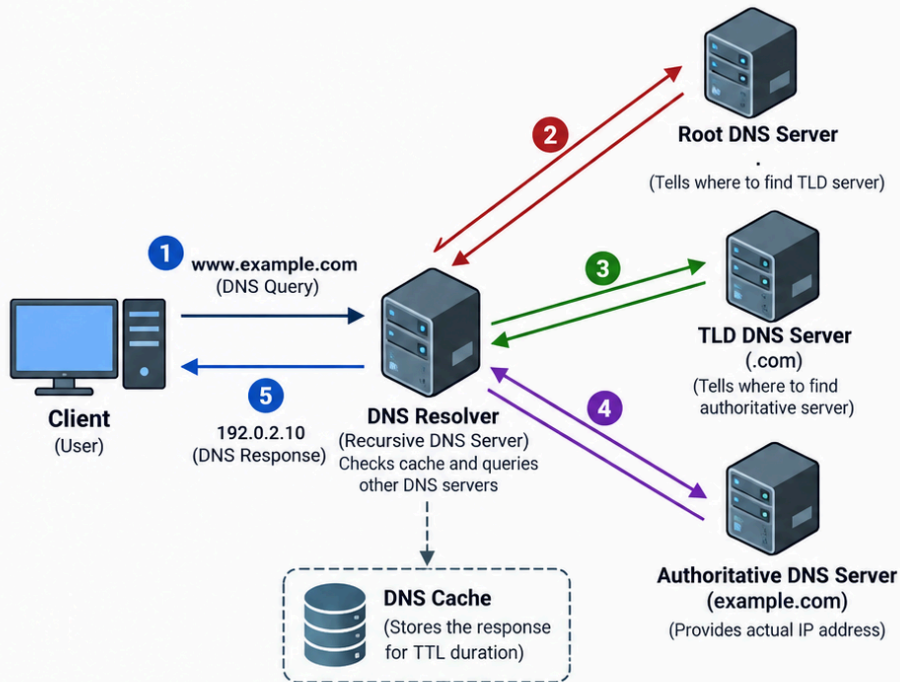
The resolver performs the lookup and may use its **DNS cache** before contacting other DNS servers.

2.2 DNS Resolution Process

The complete resolution process is shown below.

DNS Resolution Process

Example: Client requests `www.example.com`



Key Points

- DNS uses port 53 (UDP, and TCP when required).
- The resolver checks cache before performing a lookup.
- DNS follows a hierarchical structure: Root → TLD → Authoritative.
- Responses are cached based on TTL (Time To Live).
- In our Debian setup, BIND9 acts as an authoritative DNS server for `company.local`.

Process Summary

- Client sends a DNS query.
- Resolver checks its cache.
- If the answer is not cached, the resolver contacts the required DNS hierarchy.
- Authoritative DNS provides the final record.
- Resolver returns the answer to the client and caches it according to **TTL**.

2.3 Important DNS Components

Component	Role
DNS Client	Sends DNS queries
DNS Resolver	Performs lookup on behalf of the client
Root DNS	Directs queries toward the correct TLD
TLD DNS	Directs queries toward the authoritative server
Authoritative DNS	Provides the official DNS record

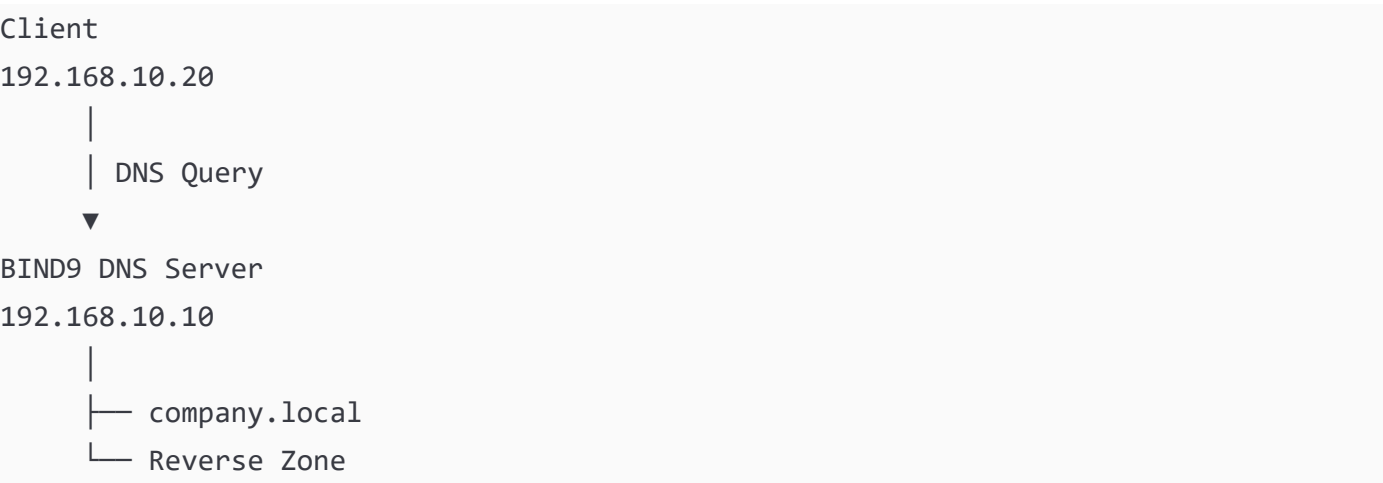
Component	Role
DNS Cache	Stores previous responses according to TTL

Recursive vs Authoritative

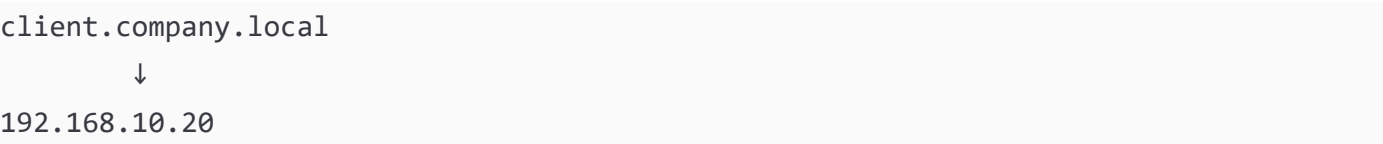
Recursive DNS	Authoritative DNS
Finds the answer on behalf of the client	Holds the official records for a zone
Usually checks cache first	Answers for its configured zone
May query other DNS servers	Does not need to search the hierarchy for its own zone

2.4 Internal DNS Architecture

In our Debian/BIND9 environment, the architecture is simpler because BIND9 directly manages the local DNS zone.



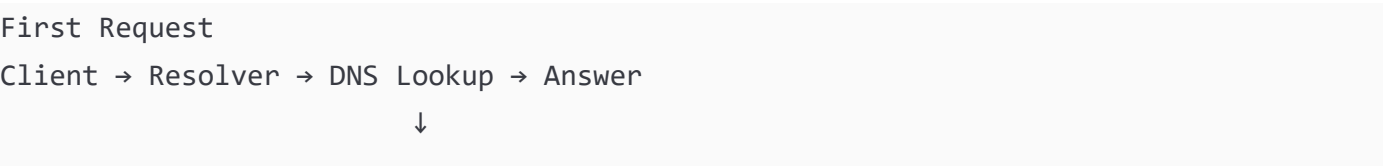
For example:



BIND9 reads the configured **forward and reverse zone files** and provides the corresponding DNS records.

2.5 DNS Caching

DNS responses are temporarily stored by the resolver.



Cache

Next Request

Client → Resolver → Cached Answer

The **TTL (Time To Live)** determines how long a response can remain cached.

DNS uses a hierarchical resolution process, while BIND9 can act as an authoritative DNS server for our internal zones.

Section 3: Prerequisites & BIND9 Installation

3.1 Lab Environment

System	IP Address	Role
DNS Server	192.168.10.10	BIND9 DNS Server
Client	192.168.10.20	DNS Testing
Domain	company.local	Internal DNS Zone

Both systems should have **static IP addresses** and basic network connectivity.

3.2 Install BIND9

On the DNS server:

```
sudo apt update
sudo apt install bind9 bind9-dnsutils -y
```

Optional documentation:

```
sudo apt install bind9-doc -y
```

- bind9 → DNS server
- bind9-dnsutils → DNS testing utilities such as nslookup
- bind9-doc → optional documentation

3.3 Verify BIND9

Check service status:

```
sudo systemctl status bind9
```

Enable and start the service:

```
sudo systemctl enable --now bind9
```

Check that DNS is listening on port 53:

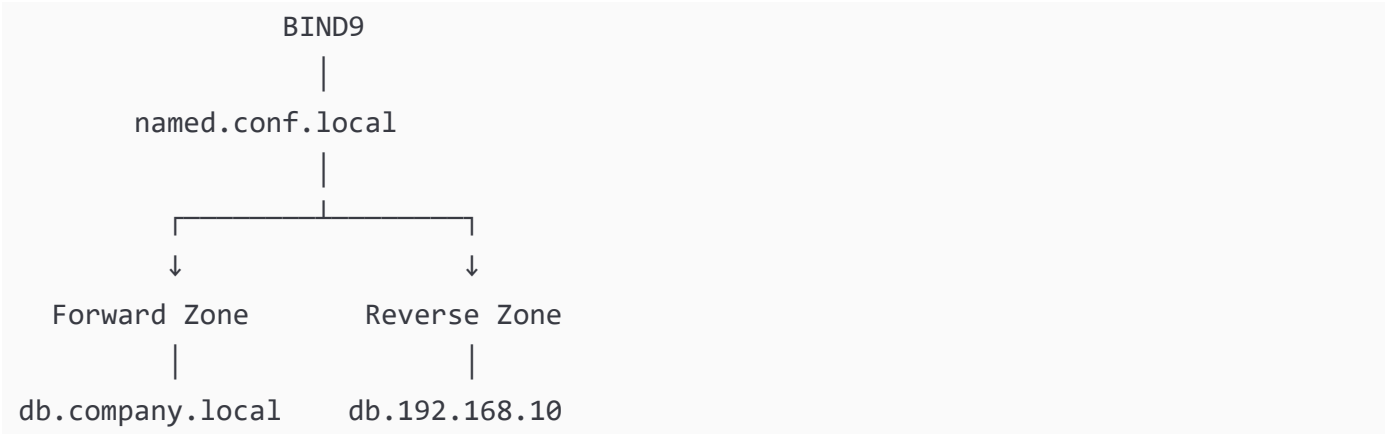
```
sudo ss -ltnup | grep :53
```

DNS primarily uses **port 53**.

3.4 Important BIND9 Files

Path	Purpose
/etc/bind/	BIND9 configuration directory
/etc/bind/named.conf	Main BIND configuration
/etc/bind/named.conf.local	Custom zone declarations
/etc/bind/db.*	DNS zone files

Our configuration will mainly follow:



The zone declarations and separate forward/reverse zone files are part of the configuration approach in the provided notes.

3.5 Basic Connectivity Check

From the DNS server:

```
ping -c 4 192.168.10.20
```

From the client:

```
ping -c 4 192.168.10.10
```

The machines must be able to communicate before proceeding with DNS testing.

Section 4: BIND9 Configuration

4.1 Configure Zone Declarations

BIND9 uses `/etc/bind/named.conf.local` to define the DNS zones managed by the server.

Open the file:

```
sudo nano /etc/bind/named.conf.local
```

Forward Zone

```
zone "company.local" {
    type master;
    file "/etc/bind/db.company.local";
    allow-update { none; };
};
```

Reverse Zone

For network `192.168.10.0/24`:

```
zone "10.168.192.in-addr.arpa" {
    type master;
    file "/etc/bind/db.192.168.10";
    allow-update { none; };
};
```

Important Parameters

Parameter	Meaning
<code>zone</code>	DNS zone name
<code>type master</code>	This server is authoritative for the zone
<code>file</code>	Location of the zone file
<code>allow-update { none; };</code>	Prevents dynamic DNS updates

4.2 Create Zone Files

Create the forward zone file:

```
sudo nano /etc/bind/db.company.local
```

Create the reverse zone file:

```
sudo nano /etc/bind/db.192.168.10
```

4.3 Validate BIND Configuration

Before restarting BIND9, always check the configuration:

```
sudo named-checkconf
```

No output normally means the configuration syntax is valid.

Validate the forward zone:

```
sudo named-checkzone company.local /etc/bind/db.company.local
```

Validate the reverse zone:

```
sudo named-checkzone 10.168.192.in-addr.arpa /etc/bind/db.192.168.10
```

Best practice: Validate first, then reload/restart. This prevents a syntax error from taking down a working DNS configuration.

4.4 Reload BIND9

After successful validation:

```
sudo systemctl reload bind9
```

Check the service:

```
sudo systemctl status bind9
```

If a complete restart is required:

```
sudo systemctl restart bind9
```

4.5 Configuration Workflow

Edit named.conf.local

↓

Create Zone Files

↓

named-checkconf

↓

named-checkzone

↓

Reload BIND9



Test DNS

Declare the zones in `named.conf.local`, create their zone files, validate the configuration and zones, then reload BIND9.

Section 5: Forward & Reverse Zones

5.1 Forward DNS Zone

A **forward zone** resolves a hostname to an IP address.

```
server.company.local → 192.168.10.10
client.company.local → 192.168.10.20
```

Create the zone file:

```
sudo nano /etc/bind/db.company.local
```

Example:

```
$TTL 604800

@ IN SOA server.company.local. admin.company.local. (
    1
    604800
    86400
    2419200
    604800
)

@ IN NS server.company.local.
@ IN A 192.168.10.10
server IN A 192.168.10.10
client IN A 192.168.10.20
www IN A 192.168.10.10
```

Important Records

Record	Purpose
SOA	Defines authority and zone parameters
NS	Specifies the authoritative DNS server
A	Maps hostname → IPv4 address
AAAA	Maps hostname → IPv6 address
CNAME	Creates an alias for another hostname

Note: `www` does not require a CNAME. An `A` record can directly point `www` to an IP. CNAME is used when `www` should be an alias of another hostname.

Example:

```
www IN CNAME server.company.local.
```

5.2 Understanding SOA

The SOA record contains important zone information:

```
@ IN SOA server.company.local. admin.company.local. (  
    1  
    604800  
    86400  
    2419200  
    604800  
)
```

Value	Purpose
1	Serial number; increase when zone data changes
604800	Refresh
86400	Retry
2419200	Expire
604800	Negative caching / minimum TTL

`$TTL 604800` sets the default TTL for records that do not specify another TTL.

5.3 Reverse DNS Zone

A **reverse zone** resolves an IP address back to a hostname using **PTR records**.

For:

```
192.168.10.0/24
```

the reverse zone is:

```
10.168.192.in-addr.arpa
```

Create the zone file:

```
sudo nano /etc/bind/db.192.168.10
```

Example:

```
$TTL 604800
```

```
@    IN  SOA  server.company.local. admin.company.local. (
      1
      604800
      86400
      2419200
      604800
)
```

```
@    IN  NS   server.company.local.
10   IN  PTR  server.company.local.
20   IN  PTR  client.company.local.
```

The last octet is used in the PTR record:

```
192.168.10.10 → 10 → server.company.local
```

```
192.168.10.20 → 20 → client.company.local
```

5.4 Forward vs Reverse DNS

Type	Record	Example
Forward	A	server.company.local → 192.168.10.10
Reverse	PTR	192.168.10.10 → server.company.local

Forward

Reverse

Hostname

IP Address

|

|



192.168.10.10

server.company.local

5.5 Validate the Zones

After creating the files:

```
sudo named-checkzone company.local /etc/bind/db.company.local
```

```
sudo named-checkzone 10.168.192.in-addr.arpa /etc/bind/db.192.168.10
```

A successful result should end with:

```
OK
```

Then reload BIND9:

```
sudo systemctl reload bind9
```

Forward DNS uses records such as A to resolve names to IPs, while Reverse DNS uses PTR records to resolve IPs back to names.

Section 6: Client Configuration & Testing

6.1 Configure Client DNS

The client must use the BIND9 server as its DNS resolver.

DNS Server:

```
192.168.10.10
```

Check the current resolver configuration:

```
cat /etc/resolv.conf
```

It should contain:

```
nameserver 192.168.10.10
```

6.2 Test Forward DNS

Use `nslookup`:

```
nslookup server.company.local
```

Expected result:

```
Name:      server.company.local
```

```
Address: 192.168.10.10
```

Test the client record:

```
nslookup client.company.local
```

Test using `dig`:

```
dig server.company.local
```

For a short result:

```
dig +short server.company.local
```

Expected:

```
192.168.10.10
```

6.3 Test Reverse DNS

Query the IP address:

```
nslookup 192.168.10.10
```

Or:

```
dig -x 192.168.10.10 +short
```

Expected:

```
server.company.local.
```

This confirms that the **PTR record** is working.

6.4 Test DNS Server Directly

To bypass the client's configured resolver and query BIND9 directly:

```
dig @192.168.10.10 server.company.local
```

For reverse DNS:

```
dig @192.168.10.10 -x 192.168.10.10
```

This is particularly useful when troubleshooting whether the problem is with **BIND9 or the client's resolver configuration**.

6.5 Test Web Access Using DNS

If a web server is configured on the DNS server:

```
curl http://www.company.local
```

Successful output confirms:

```
DNS Resolution → Web Server Connection
```

6.6 Basic Testing Checklist

Test	Command	Expected
Forward lookup	<code>nslookup server.company.local</code>	<code>192.168.10.10</code>
Reverse lookup	<code>nslookup 192.168.10.10</code>	<code>server.company.local</code>
Direct BIND query	<code>dig @192.168.10.10 server.company.local</code>	DNS answer
Short lookup	<code>dig +short server.company.local</code>	IP address

Test	Command	Expected
Web test	<code>curl http://www.company.local</code>	Web response

Section 7: Validation, Security

7.1 Configuration Validation

Before applying any DNS changes, validate the configuration and zone files.

Check BIND Configuration

```
sudo named-checkconf
```

Check Forward Zone

```
sudo named-checkzone company.local /etc/bind/db.company.local
```

Check Reverse Zone

```
sudo named-checkzone 10.168.192.in-addr.arpa /etc/bind/db.192.168.10
```

A successful zone check should end with:

```
OK
```

After successful validation:

```
sudo systemctl reload bind9
```

7.2 Basic DNS Security

For an internal DNS server:

- Allow queries only from required networks.
- Disable dynamic updates if they are not required.
- Keep BIND9 updated.
- Do not expose internal DNS unnecessarily to the Internet.
- Restrict firewall access to required DNS clients.

Example:

```
allow-query { 192.168.10.0/24; };
```

For a static zone without dynamic DNS:

```
allow-update { none; };
```

7.3 Validation Workflow

Edit Configuration



named-checkconf



named-checkzone



Reload BIND9



Test DNS

Best Practice: Always validate configuration and zone files before reloading BIND9.

DNS — Section 8: Troubleshooting

8.1 Troubleshooting Order

When DNS fails, check systematically:



8.2 Common Problems & Checks

Problem	Check	Command
BIND9 not running	Service	<code>sudo systemctl status bind9</code>
Configuration error	Syntax	<code>sudo named-checkconf</code>
Zone not loading	Zone syntax	<code>sudo named-checkzone ...</code>
Port 53 unavailable	Listening ports	<code>sudo ss -ltnup grep :53</code>
Name not resolving	Direct BIND query	<code>dig @192.168.10.10 server.company.local</code>
Reverse lookup fails	PTR / reverse zone	<code>dig -x 192.168.10.10</code>
Client uses wrong DNS	Resolver	<code>cat /etc/resolv.conf</code>
Unknown cause	BIND logs	<code>sudo journalctl -u bind9 -n 50</code>

8.3 Common Administrator Mistakes

1. Missing Trailing Dot

Correct:

```
server.company.local.
```

Incorrect:

```
server.company.local
```

The trailing `.` represents the complete DNS name.

2. Forgetting the Zone Serial Number

After modifying a zone, increase the serial:

```
1
```

to:

```
2
```

This is important when zone changes need to be recognized by DNS systems using the zone's serial.

3. Incorrect Reverse Zone

For:

```
192.168.10.0/24
```

the reverse zone must be:

```
10.168.192.in-addr.arpa
```

not:

```
192.168.10.in-addr.arpa
```

4. DNS Works on Server but Not Client

First test BIND directly:

```
dig @192.168.10.10 server.company.local
```

If this works, check the client's DNS resolver configuration.

5. Restarting Without Validation

Avoid immediately restarting after editing configuration.

Use:

```
sudo named-checkconf  
sudo named-checkzone company.local /etc/bind/db.company.local
```

```
sudo systemctl reload bind9
```

8.4 `/etc/resolv.conf` Problem

Check the client's resolver:

```
cat /etc/resolv.conf
```

It should ultimately use:

```
nameserver 192.168.10.10
```

If `/etc/resolv.conf` is a symlink or automatically managed, identify the service controlling it before changing it.

Do not use `chattr +i` as a normal solution.

8.5 Check BIND9 Logs

For recent errors:

```
sudo journalctl -u bind9 -n 50
```

For live monitoring:

```
sudo journalctl -u bind9 -f
```

Look for:

- Configuration syntax errors
 - Zone loading failures
 - Permission problems
 - Invalid DNS records
 - Service startup failures
-

8.6 Quick Troubleshooting Flow

DNS Query Fails

↓

Can client reach DNS server?

↓

Is BIND9 running?

↓

Is port 53 listening?

↓

named-checkconf

↓

named-checkzone

↓

Check /etc/resolv.conf

↓

dig @DNS_SERVER DOMAIN

↓

Check BIND9 logs

Section 9: Real-World Scenario

9.1 Office Internal DNS

Scenario

A company has an internal network:

Network: 192.168.10.0/24

The company wants employees to access internal services using **names instead of IP addresses**.

Device	IP Address	DNS Name
DNS Server	192.168.10.10	server.company.local
Web Server	192.168.10.10	www.company.local
Client PC	192.168.10.20	client.company.local

BIND9 is installed on 192.168.10.10.

9.2 Required DNS Records

Forward Zone

server.company.local → 192.168.10.10
www.company.local → 192.168.10.10
client.company.local → 192.168.10.20

Reverse Zone

192.168.10.10 → server.company.local
192.168.10.20 → client.company.local

9.3 Working Flow



```
192.168.10.10
```

```
|
```



```
Internal Web Server
```

9.4 Implementation

1. Configure the client DNS

```
nameserver 192.168.10.10
```

2. Test name resolution

```
nslookup www.company.local
```

Expected:

```
Address: 192.168.10.10
```

3. Test the application

```
curl http://www.company.local
```

If the webpage is returned, the complete path is working:

```
Client → DNS Resolution → IP Address → Web Server
```

9.5 Reverse DNS Verification

```
dig -x 192.168.10.10 +short
```

Expected:

```
server.company.local.
```

This confirms that the **PTR record** is correctly configured.

DNS — Section 10: Quick Reference & Viva

10.1 Important Commands

Task	Command
Install BIND9	<code>sudo apt install bind9 bind9-dnsutils -y</code>
Check service	<code>sudo systemctl status bind9</code>
Validate config	<code>sudo named-checkconf</code>
Validate zone	<code>sudo named-checkzone ZONE FILE</code>
Reload BIND9	<code>sudo systemctl reload bind9</code>
Check port 53	<code>sudo ss -lntup grep :53</code>
Forward lookup	<code>dig server.company.local</code>
Reverse lookup	<code>dig -x 192.168.10.10</code>
Query specific DNS	<code>dig @192.168.10.10 server.company.local</code>
Check logs	<code>sudo journalctl -u bind9 -n 50</code>

10.2 Important Files

/etc/bind/named.conf
/etc/bind/named.conf.local
/etc/bind/db.company.local
/etc/bind/db.192.168.10

10.3 Important DNS Records

Record	Function
A	Hostname → IPv4
AAAA	Hostname → IPv6
PTR	IP → Hostname
CNAME	Hostname alias
NS	Authoritative name server
SOA	Zone authority information
MX	Mail server
TXT	Text/additional information

10.4 Viva Questions

Q1. What is DNS?

DNS translates domain/hostnames into IP addresses and supports reverse resolution.

Q2. What is BIND9?

BIND9 is DNS server software commonly used on Linux.

Q3. Which port does DNS use?

Port **53**, using UDP and TCP depending on the operation.

Q4. What is a forward zone?

A zone used for **hostname** → **IP** resolution.

Q5. What is a reverse zone?

A zone used for **IP** → **hostname** resolution.

Q6. Which record is used for IPv4 address mapping?

A.

Q7. Which record is used for reverse DNS?

PTR.

Q8. What is the purpose of `named.conf.local`?

It contains custom DNS zone declarations.

Q9. Why is `named-checkconf` used?

To check BIND configuration syntax.

Q10. Why is `named-checkzone` used?

To validate the syntax and structure of a DNS zone file.

Q11. What does TTL mean?

Time To Live — it controls how long DNS information can be cached.

Q12. Why should the zone serial number be increased after changes?

To indicate that the zone data has been updated.

Q13. What is the difference between recursive and authoritative DNS?

A recursive server finds answers on behalf of clients; an authoritative server provides official records for its configured zone.